

Tarea corta 2

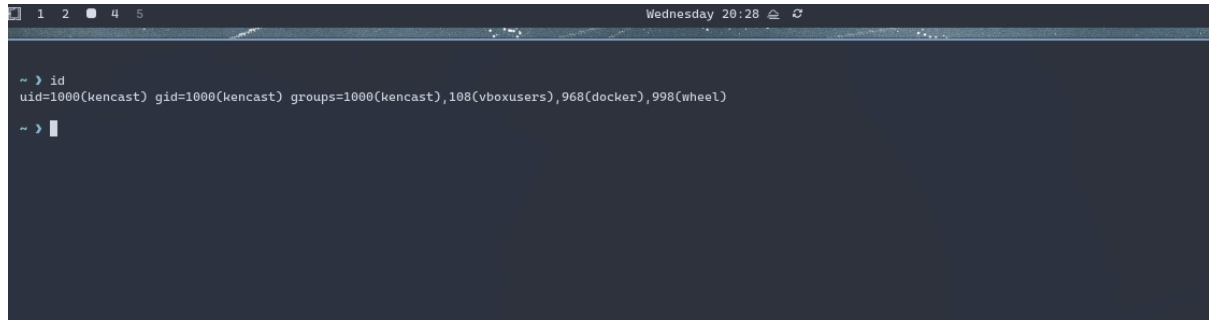
Kener Adiel Castillo

2023179846

Seguridad del software

Fase 1.

Uso Arch Linux, en el entorno:



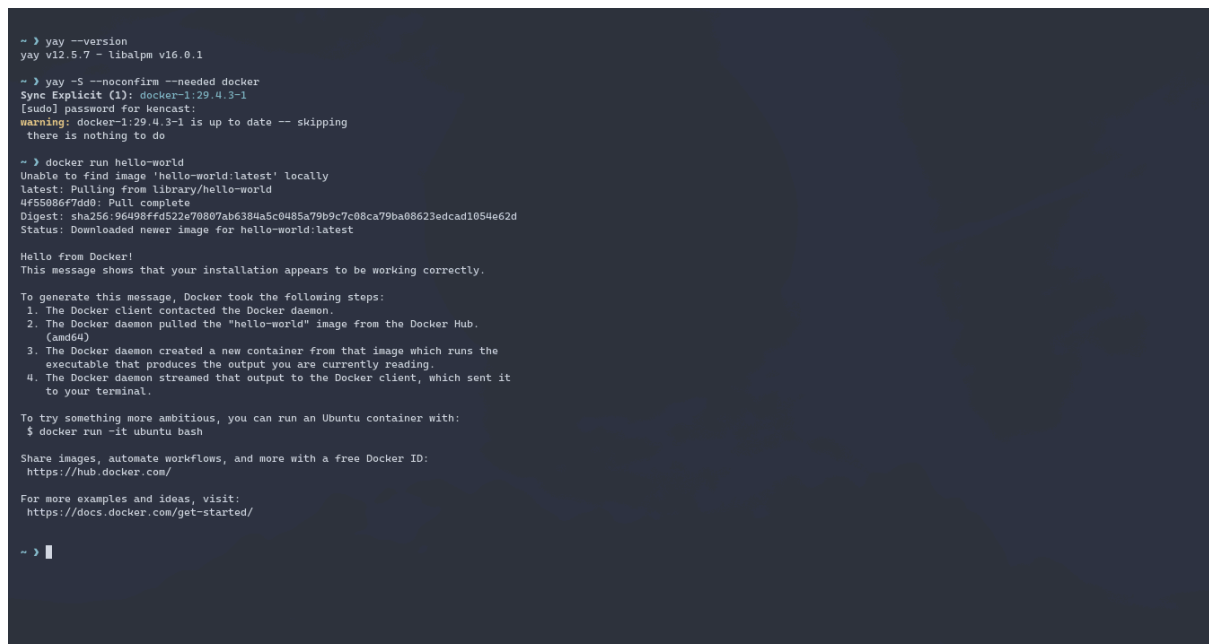
```
~ > id
uid=1000(kencast) gid=1000(kencast) groups=1000(kencast),108(vboxusers),968(docker),998(wheel)

~ > |
```

Docker se instaló con el siguiente comando:

```
yay -S --noconfirm --needed docker
```

Evidencia con `docker run hello-world`:



```
~ > yay --version
yay v12.5.7 - libalpm v16.0.1

~ > yay -S --noconfirm --needed docker
sync Explicit (1): docker-1:29.4.3-1
[sudo] password for kencast:
warning: docker-1:29.4.3-1 is up to date -- skipping
there is nothing to do

~ > docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55986f7d90: Pull complete
Digest: sha256:96498ffd522e70807ab6384a5c0485a79b9c7c08ca79ba08623edcad1854e62d
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

~ > |
```

Fase 2.

Pull de Alpine y ejecución de comando 'ls':

```

~ > docker pull alpine
Using default tag: latest
latest: Pulling from library/alpine
9b70e313681f: Pull complete
Digest: sha256:a2d49ea686c2adfe3c992e47dc3b5e7fa6e6b5055609400dc2acaeb241c829f4
Status: Downloaded newer image for alpine:latest
docker.io/library/alpine:latest

~ > docker images

IMAGE                ID                DISK USAGE    CONTENT SIZE    EXTRA
alpine:latest        772bee4540c4      8.45MB        0B
hello-world:latest   e2ac70e7319a      10.1kB        0B      U

~ > docker run alpine ls -l
total 0
drwxr-xr-x 1 root root      858 Jun  9 14:13 bin
drwxr-xr-x 5 root root     340 Jun 10 21:13 dev
drwxr-xr-x 1 root root      56 Jun 10 21:13 etc
drwxr-xr-x 1 root root       0 Jun  9 14:13 home
drwxr-xr-x 1 root root     146 Jun  9 14:13 lib
drwxr-xr-x 1 root root      28 Jun  9 14:13 media
drwxr-xr-x 1 root root       0 Jun  9 14:13 mnt
drwxr-xr-x 1 root root       0 Jun  9 14:13 opt
dr-xr-xr-x 355 root root     0 Jun 10 21:13 proc
drwx----- 1 root root       0 Jun  9 14:13 root
drwxr-xr-x 1 root root       8 Jun  9 14:13 run
drwxr-xr-x 1 root root     810 Jun  9 14:13 sbin
drwxr-xr-x 1 root root       0 Jun  9 14:13 srv
dr-xr-xr-x 13 root root     0 Jun 10 21:13 sys
drwxrwxrwt 1 root root       0 Jun  9 14:13 tmp
drwxr-xr-x 1 root root      40 Jun  9 14:13 usr
drwxr-xr-x 1 root root      86 Jun  9 14:13 var

```

Corriendo el contenedor:

```

~ X docker run -it alpine sh
/ # ^C

/ # exit

~ X docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS          PORTS          NAMES
41195a53dcfd   alpine    "sh"                    About a minute ago   Exited (130) 5 seconds ago
10d783865414   alpine    "/bin/sh."              4 minutes ago       Created
96c4c25981e5   alpine    "ls -l"                  6 minutes ago       Exited (0) 6 minutes ago
5f545b97455c   hello-world "/hello"                13 minutes ago      Exited (0) 13 minutes ago
nervous_goodall

```

Preguntas:

(1) Ejecute `docker run alpine /bin/sh` sin las banderas `-i` ni `-t` y documente con captura de pantalla lo que ocurre. ¿Por qué no sucede nada visible (no aparece un prompt interactivo)? Explique el rol de las banderas `-i` (interactive) y `-t` (pseudo-TTY) y por qué su ausencia hace que el contenedor termine inmediatamente.

```

~ > docker run alpine sh

~ > docker run -t alpine sh
/ #

^C^C

/ # exit
exit

```

No poner las banderas hace que el comando termine y no haga nada.

Al ejecutar con la bandera **-t** se abre una terminal; sin embargo, no permite interactuar con esta, como se observa en la imagen.

```

~ > docker run -i alpine sh
l
sh: l: not found
o
sh: o: not found

```

Al correrlo solo con la bandera **-i** el programa se queda corriendo, pero no permite hacer nada. Por lo que el rol de la bandera **-t** es abrir una terminal y **-i** poder interactuar (input). La ausencia de estas banderas ocasiona que el contenedor termine inmediatamente porque este no tiene nada que hacer o ejecutar, al no ejecutar una terminal (al correr 'docker ps' no sale nada).

(2) Inicie una sesión interactiva con `docker run -it alpine /bin/sh`.
Identifique con qué usuario está corriendo el shell dentro del contenedor (use el comando `id` o `whoami`). ¿Qué implicaciones de seguridad tiene que ese sea el usuario por defecto? Incluya captura de pantalla.

```

~ > docker run -it alpine sh
/ # id
uid=0(root) gid=0(root) groups=0(root),0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),10(wheel),11(floppy),20(dialout),26(tape),27(video)
/ # ls
bin    dev    etc    home  lib    media mnt    opt    proc  root  run    sbin  srv    sys    tmp    usr    var
/ #

```

El usuario que está corriendo por defecto es el root. Esto tiene implicaciones de seguridad, ya que alguien con acceso al contenedor tendría control total sobre el sistema que se está ejecutando, lo cual no es así en un sistema real (no en contenedor) como Linux.

(3) Dentro de la sesión interactiva del punto anterior, cree un archivo con `touch /tmp/miarchivo` y verifique su existencia con `ls -lah`

/tmp/miarchivo. Documente con captura el resultado.

```
~ > docker run -it alpine sh
/ # touch tmp/miarchivo
/ # ls -lah /tmp/miarchivo
-rw-r--r--  1 root    root          0 Jun 11 01:30 /tmp/miarchivo
/ # █
```

Se muestra archivo creado con root.

(4) Termine la sesión interactiva con exit, vuelva a iniciar una sesión con `docker run -it alpine /bin/sh` y verifique si el archivo `/tmp/miarchivo` se encuentra en el directorio `/tmp`. ¿Por qué considera que no está? Explique el concepto de efemeralidad del sistema de archivos del contenedor y la diferencia entre el ciclo de vida de la imagen y el ciclo de vida del contenedor.

El directorio se encuentra vacío al revisarlo en una nueva sesión. Considero que es porque no se tiene la persistencia activada entre sesiones o porque el sistema de archivos está diseñado sin persistencia.

El concepto de efemeralidad del sistema de archivos del contenedor trata de que los cambios o los datos guardados en un contenedor son temporales, por lo que no se guardan al volver a ejecutar o eliminar el contenedor.

La diferencia entre el ciclo de vida de la imagen y el contenedor es que la imagen es como un archivo que define o contiene toda la app, la cual es única y es compartida o publicada, mientras que el contenedor es una instancia de la imagen, por lo que pueden haber muchos contenedores de la misma imagen y la vida de estos se considera temporal.

Fase 3.

Preguntas:

(5) Ejecute `docker run -d dockersamples/static-site` y describa qué sucedió con el comando. ¿Qué hace la bandera `-d`? ¿Por qué el comando devolvió el control de la terminal inmediatamente? ¿El servicio quedó accesible desde el navegador en ese momento? Justifique.

El comando devuelve un string alfanumerico y retorna el control de la terminal. La bandera `'-d'` le dice corra el contenedor en segundo plano, por lo que el comando devuelve el control de la terminal. El contenedor está disponible ya que al ejecutar el comando `'docker ps'` aparece el contenedor creado corriendo, pero no es accesible desde el navegador porque no se expusieron puertos.

```

~ > docker run -d dockersamples/static-site
cd5ea3b748713127e4775becef52249b7402d6f46f3e08224b1eaa5a8cbcdddf

~ > docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED        STATUS        PORTS                NAMES
cd5ea3b74871   dockersamples/static-site  "/bin/sh -c 'cd /usr..."  2 minutes ago  Up 2 minutes  80/tcp, 443/tcp      crazy_hypatia
~ > █

```

(6) Ejecute `docker run --name misitio -e AUTHOR="su nombre" -d -P dockersamples/static-site`. Describa con precisión qué hace cada uno de los siguientes parámetros: `--name`, `-e`, `-d` y `-P`. Documente la salida con captura de pantalla.

El parámetro `--name NOMBRE` lo que hace es ponerle al contenedor el nombre especificado por en el comando, si no se pone se asigna un nombre random.

El parámetro `-e DATO=X` lo que hace es definir una variable de entorno accesible desde el contenedor.

El parámetro `-d` lo que hace es ejecutar el contenedor en segundo plano (no se abre terminal o algo).

El parámetro `-P` permite la exposición de puertos (asignados de manera automática por Docker) para comunicación entre el host y el contenedor.

Salida:

```

~ > docker run --name misitio -e AUTHOR="KENER" -d -P dockersamples/static-site
ef7121ce36a9d773b78a3340dab67e4ff7bcefb5fafdb77c3627235da529de8

~ > docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED        STATUS        PORTS
ef7121ce36a9   dockersamples/static-site  "/bin/sh -c 'cd /usr..."  4 seconds ago  Up 4 seconds  0.0.0.0:32770->80/tcp, [::]:32770->80/tcp, 0.0.0.0:32771->443/tcp, [::]:32771->443/tcp
misitio

```

(7) ¿Cuál es la principal diferencia entre la bandera `-P` (mayúscula) y `-p` (minúscula) al exponer puertos? Demuestre con un segundo contenedor levantado con `-p 8888:80` y compare la salida de `docker port` entre ambos contenedores.

La bandera `-P` lo que hace es que docker asigna automáticamente los puertos de forma arbitraria, mientras que `-p` lo hace de forma explícita `PUERTO_HOST:PUERTO_CONTENEDOR`.

```

~ > docker run --name prueba -d -p 8888:80 dockersamples/static-site
c7967d1bf5531e7d9e8d774c9d2e7278b5584a9cb5b8aae861f7885b76adcf25

~ > docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED        STATUS        PORTS
c7967d1bf553   dockersamples/static-site  "/bin/sh -c 'cd /usr..."  3 seconds ago  Up 3 seconds  443/tcp, 0.0.0.0:8888->80/tcp, [::]:8888->80/tcp
prueba
ef7121ce36a9   dockersamples/static-site  "/bin/sh -c 'cd /usr..."  9 minutes ago  Up 9 minutes  0.0.0.0:32770->80/tcp, [::]:32770->80/tcp, 0.0.0.0:32771->443/tcp, [::]:32771->443/tcp
misitio

```

La diferencia es para **misitio** los puertos van: `[::]:32770->80/tcp` y `[::]:32771->443/tcp`. Mientras que para **prueba** `[::]:8888->80/tcp` como se especificó.

(8) Al eliminar contenedores con `docker rm -f sitio2`, ¿qué hace exactamente la bandera `-f`? ¿En qué se diferencia de hacer `docker stop` seguido de `docker rm`? Mencione un escenario en el que prefiera el

enfoque de dos pasos sobre el forzado.

```
~ > docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
c7967d1bf553   dockersamples/static-site          "/bin/sh -c 'cd /usr..." 3 seconds ago  Up 3 seconds  443/tcp, 0.0.0.0:8888->80/tcp, [::]:8888->80/tcp
ef7121ce36a9   dockersamples/static-site          "/bin/sh -c 'cd /usr..." 9 minutes ago  Up 9 minutes  0.0.0.0:32770->80/tcp, [::]:32770->80/tcp, 0.0.0.0:32771->443/tcp, [::]:32771->443/tcp
cd5ea3b74871   dockersamples/static-site          "/bin/sh -c 'cd /usr..." 41 minutes ago Up 41 minutes  80/tcp, 443/tcp
crazy_hypatia

~ > docker rm -f prueba
prueba

~ > docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
ef7121ce36a9   dockersamples/static-site          "/bin/sh -c 'cd /usr..." 26 minutes ago Up 26 minutes  0.0.0.0:32770->80/tcp, [::]:32770->80/tcp, 0.0.0.0:32771->443/tcp, [::]:32771->443/tcp
cd5ea3b74871   dockersamples/static-site          "/bin/sh -c 'cd /usr..." 58 minutes ago Up 58 minutes  80/tcp, 443/tcp
crazy_hypatia
```

La bandera `-f` fuerza a que se elimine el contenedor aunque esté corriendo. Se diferencia que `docker stop` termina la ejecución y `docker rm` elimina el contenedor una vez detenido. Un escenario en el se puede preferir el enfoque de dos pasos es cuando se quiera parar el contenedor y después volver a ejecutarlo para más adelante eliminarlo.

Fase 4.

(9) Ingrese a <https://hub.docker.com/> y liste los 20 contenedores más populares (puede ordenar por descargas/pulls). Adicionalmente, identifique un contenedor que sea de su interés profesional o personal y justifique brevemente por qué lo seleccionó.

Filter by (3) [Clear All](#) 1 - 30 of 11,871 available images. Pull count

Products ▼ DOCKER OFFICIAL IMAGE VERIFIED PUBLISHER IMAGES

- ☒ Images
- ☐ Helm Charts
- ☐ AI Models
- ☐ Compose
- ☐ Extensions
- ☐ Plugins

Show less

Trusted content ▼

- ☐ Docker Hardened Image
- ☒ Docker Official Image
- ☒ Verified Publisher
- ☐ Sponsored OSS

IMAGE + 1 MORE

memcached Docker Official Image

Free & open source, high-performance, distributed memory object caching system.

8h 1B+ 2.5K

IMAGE + 1 MORE

nginx Docker Official Image

Official build of Nginx.

17d 1B+ 10K+

IMAGE + 1 MORE

busybox Docker Official Image

Busybox base image.

18d 1B+ 3.5K

IMAGE

alpine Docker Official Image

A minimal Docker image based on Alpine Linux with a complete package index and only 5 MB in size!

1d 1B+ 10K+

IMAGE

datadog/agent Datadog

Docker container for the new Datadog Agent

1d 1B+ 172

IMAGE + 1 MORE

redis Docker Official Image

Redis is the world's fastest data platform for caching, vector search, and NoSQL databases.

1h 1B+ 10K+

IMAGE + 1 MORE

1. memcached
2. nginx
3. busybox
4. alpine
5. datadog/agent
6. redis
7. postgres
8. ubuntu
9. python

10. node
11. grafana/grafana
12. mysql
13. mongo
14. grafana/loki
15. httpd
16. timberio/vector
17. rabbitmq
18. rancher/rancher-agent
19. newrelic/infrastructure-bundle
20. traefik

Seleccionado:

Redis: es la plataforma de datos más rápida para cache, búsqueda de vectores y base de datos NoSQL. Elegí esta imagen porque me parecen llamativas las funciones que tiene y pueden ser muy útiles en diferentes proyectos.

(10) Basado en el contenedor de su interés del punto 9, ejecute docker search para identificar al menos cinco contenedores relacionados disponibles en el registro. Describa brevemente cada uno y explique qué lo diferencia del contenedor que eligió originalmente.

```
~ > docker search redis
NAME                DESCRIPTION                STARS     OFFICIAL
mcp/redis            Access to Redis database operations. 14
redis                Redis is the world's fastest data platform f... 13584    [OK]
redis/redis-stack-server  redis-stack-server installs a Redis server w... 102
redis/redis-stack     redis-stack installs a Redis server with add... 160
redis/redisinsight    Redis Insight - our best official GUI for Re... 51
redislabs/redis       Clustered in-memory database engine compatib... 46
redis/rdi-operator    0
redis/rdi-collector-initializer  Init container for RDI Collector 0
redis/rdi-api         0
redis/rdi-monitor     0
redis/rdi-collector-api 0
redis/rdi-processor    1
redis/rdi-cli          0
redis/rdi-flink-processor 0
redis/reloader         0
redis/rdi-metrics-aggregator 0
redis/rdi-flink-collector 0
circleci/redis        CircleCI images for Redis 18
redis/sidekiq-benchmark  Sidekiq load benchmark in Rust - throughput ... 0
bitnami/redis         Bitnami Helm chart for Redis(R) 37
redis-benchmark-go     redis-benchmark utility in go. 1
redis/vector-db-benchmark  Framework for benchmarking vector search eng... 1
redis/redis-di-cli     0
cimg/redis            2
redis/debezium-server  0
```

mcp/redis: acceso a las operaciones de bases de datos de redis para modelos de IA. Lo diferencia de la imagen original en que este es un servidor MCP para la comunicación de Redis con agentes. Imagen hija y no oficial de Redis.

redis/redis-stack-server: instala un servidor de Redis con capacidades adicionales de bases de datos. Es una imagen hija de Redis con capacidades adicionales.

redis/redis-Insight: herramienta para visualizar e interactuar con datos. Se diferencia en que esta es una herramienta secundaria para el análisis.

redis/redis-stack: Igual que redis-stack-server, pero incluye RedisInsight. Es una imagen hija de Redis con capacidades adicionales.

redislabs/redis: versión empresarial y mejorada de Redis, mejor rendimiento.

Fase 5.

Creación de imagen y ejecución.

```
~/prueba > ls
Permissions Size User Date Modified Name
drwxr-xr-x - kencast 12 Jun 14:10 templates
-rw-r--r-- 200 kencast 12 Jun 15:01 app.py
-rw-r--r-- 351 kencast 12 Jun 15:23 Dockerfile
-rw-r--r-- 6 kencast 12 Jun 14:09 requirements.txt

~/prueba > |
```

```
# base image
FROM python

# Set your working directory
WORKDIR /var/www/

# Copy the necessary files
COPY ./app.py ./app.py
COPY ./requirements.txt ./requirements.txt
COPY ./templates/index.html ./templates/index.html

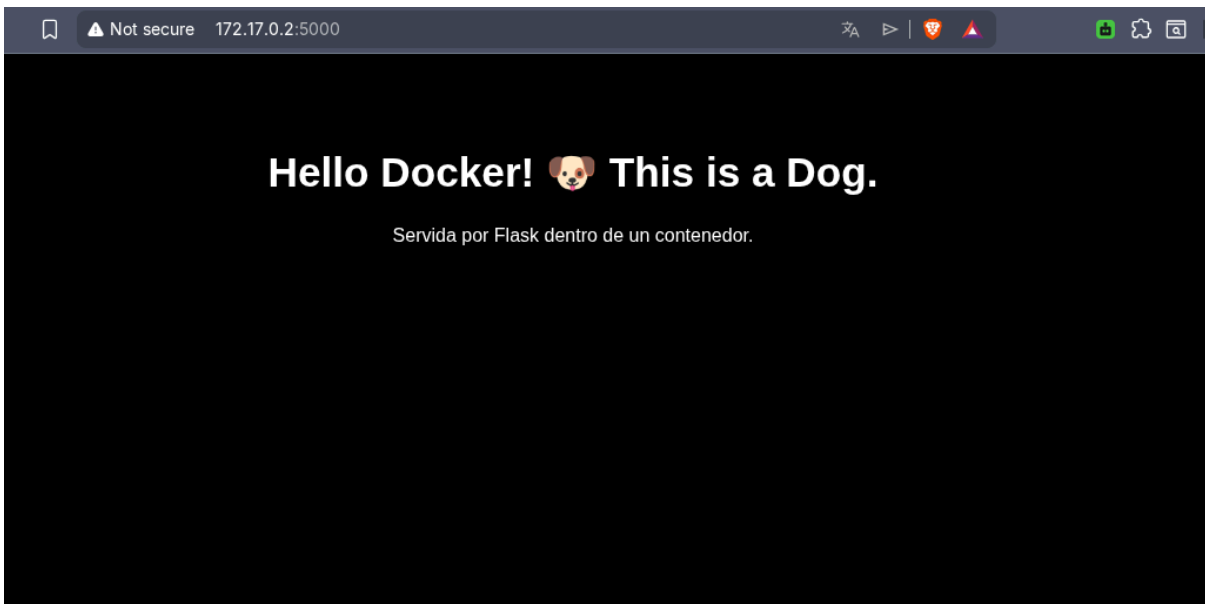
# Install the necessary packages
RUN pip install -r /var/www/requirements.txt

EXPOSE 5000
# Run the app
CMD [python, app.py]
```



```
~/prueba > docker build -t kener/myfirstapp .
[*] Building 7.5s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 390B
=> [internal] load metadata for docker.io/library/python:latest
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/6] FROM docker.io/library/python:latest@sha256:6f473f84b09fccf41d4875e19e9e2796b59d6b3c722463d2963384a43e59f39
=> [internal] load build context
=> => transferring context: 567B
=> CACHED [2/6] WORKDIR /var/www/
=> [3/6] COPY ./app.py ./app.py
=> [4/6] COPY ./requirements.txt ./requirements.txt
=> [5/6] COPY ./templates/index.html ./templates/index.html
=> [6/6] RUN pip install -r /var/www/requirements.txt
=> exporting to image
=> => exporting layers
=> => writing image sha256:7da9bf99da324fd3045afc4d7a237a43a817539be6dba171969ea52d017f0064
=> => naming to docker.io/kener/myfirstapp

~/prueba > docker run -p 8000:5000 --name app kener/myfirstapp
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.2:5000
Press CTRL+C to quit
172.17.0.1 - - [12/Jun/2026 21:24:17] "GET / HTTP/1.1" 200 -
```



(11) Una vez construida y ejecutada la imagen

YOUR_USERNAME/myfirstapp, realice una modificación en el archivo templates/index.html cambiando la palabra "Dog" por "Cat". Vuelva a lanzar el contenedor y describa con detalle cuáles tareas o problemas adicionales tuvo que realizar para que el cambio se reflejara en el navegador. ¿Tuvo que reconstruir la imagen? ¿Tuvo que detener y volver a lanzar el contenedor?

Hello Docker! 🐶 This is a Cat.

Servida por Flask dentro de un contenedor.

Tuve que hacer el cambio, borrar el contenedor y la imagen. Volver a reconstruir la imagen y lanzar el contenedor para que el cambio se reflejara.

(12) Repita el ejercicio anterior, pero esta vez monte un volumen (con la bandera `-v` o `--mount`) que asocie el directorio `templates` del equipo anfitrión con la ruta correspondiente dentro del contenedor. Modifique el estilo de `background` del `HTML` de color negro a blanco. ¿Qué diferencia hubo en relación al cambio del punto 11 en términos de pasos requeridos y tiempo? Reflexione sobre las implicaciones para flujos de desarrollo iterativo.

```
~/prueba X docker build -t kener/myfirstapp .
[+] Building 5.8s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 425B
=> [internal] load metadata for docker.io/library/python:latest
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/6] FROM docker.io/library/python:latest@sha256:6f473f84b09fccf411d4875e19e9e2796b59d6b3c722463d2963384a43e59f39
=> [internal] load build context
=> => transferring context: 63B
=> CACHED [2/6] WORKDIR /var/www/
=> [3/6] RUN mkdir -p /var/www/templates
=> [4/6] COPY ./app.py ./app.py
=> [5/6] COPY ./requirements.txt ./requirements.txt
=> [6/6] RUN pip install -r /var/www/requirements.txt
=> exporting to image
=> => exporting layers
=> => writing image sha256:713e5a6f411254100f75c78ca957633c0315cb7a03e85c6597246326b85b0d9d
=> => naming to docker.io/kener/myfirstapp

~/prueba > docker run --rm -v "$(pwd)/templates:/var/www/templates" -p 8000:5000 --name app kener/myfirstapp
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.2:5000
Press CTRL+C to quit
172.17.0.1 - - [14/Jun/2026 16:28:24] "GET / HTTP/1.1" 200 -
^C
~/prueba > docker ps -a
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS        NAMES
~/prueba > n templates/index.html

~/prueba > docker run --rm -v "$(pwd)/templates:/var/www/templates" -p 8000:5000 --name app kener/myfirstapp
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.2:5000
Press CTRL+C to quit
172.17.0.1 - - [14/Jun/2026 16:29:49] "GET / HTTP/1.1" 200 -
```



Una vez construida la imagen, solo hay necesidad de correr el contenedor. Cualquier cambio al archivo “index.htm” se muestra al correr el contenedor. Para un flujo iterativo, esto resulta mejor porque se pueden realizar cambios a los archivos sin necesidad de volver a construir la imagen, lo cual puede tardar bastante si se trata de un proyecto grande.

Subida a DockerHub:

```
~/dock > docker image ls
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
alpine:latest	772bee4540c4	8.45MB	0B	
dockersamples/static-site:latest	f589ccde7957	190MB	0B	
hello-world:latest	e2ac78e7319a	10.1kB	0B	
kencast/myfirstapp:latest	713e5a6f4112	1.13GB	0B	
kener/myfirstapp:latest	713e5a6f4112	1.13GB	0B	
mysql:8.0	6cd09145362d	799MB	0B	U
nginx:latest	936fef290c8f	161MB	0B	U

```
~/dock > docker push kencast/myfirstapp:latest
The push refers to repository [docker.io/kencast/myfirstapp]
73f03af67855: Pushed
2cabcd8fb44e0: Pushed
5b99cafb6672: Pushed
37683f74a123: Pushed
908a7d31b6bb: Pushed
a0aca403049f: Pushed
263393468c2a: Pushed
be06c251ff7c: Pushed
c8ec9b66ee7b: Pushed
3959a281940d: Pushed
4d2b45eb5dc2: Pushed
fd686728d876: Pushed
latest: digest: sha256:e9aa13693d31259ecddec90e8ef648667b04fa1d33d7b084e6cc5cf7783116a83 size: 2835

~/dock >
```

Link: <https://hub.docker.com/r/kencast/myfirstapp>

Fase 6.

```
~/dock > mkdir -p secrets

~/dock > openssl rand -base64 32 > secrets/mysql_root_password.txt

~/dock > openssl rand -base64 32 > secrets/mysql_password.txt

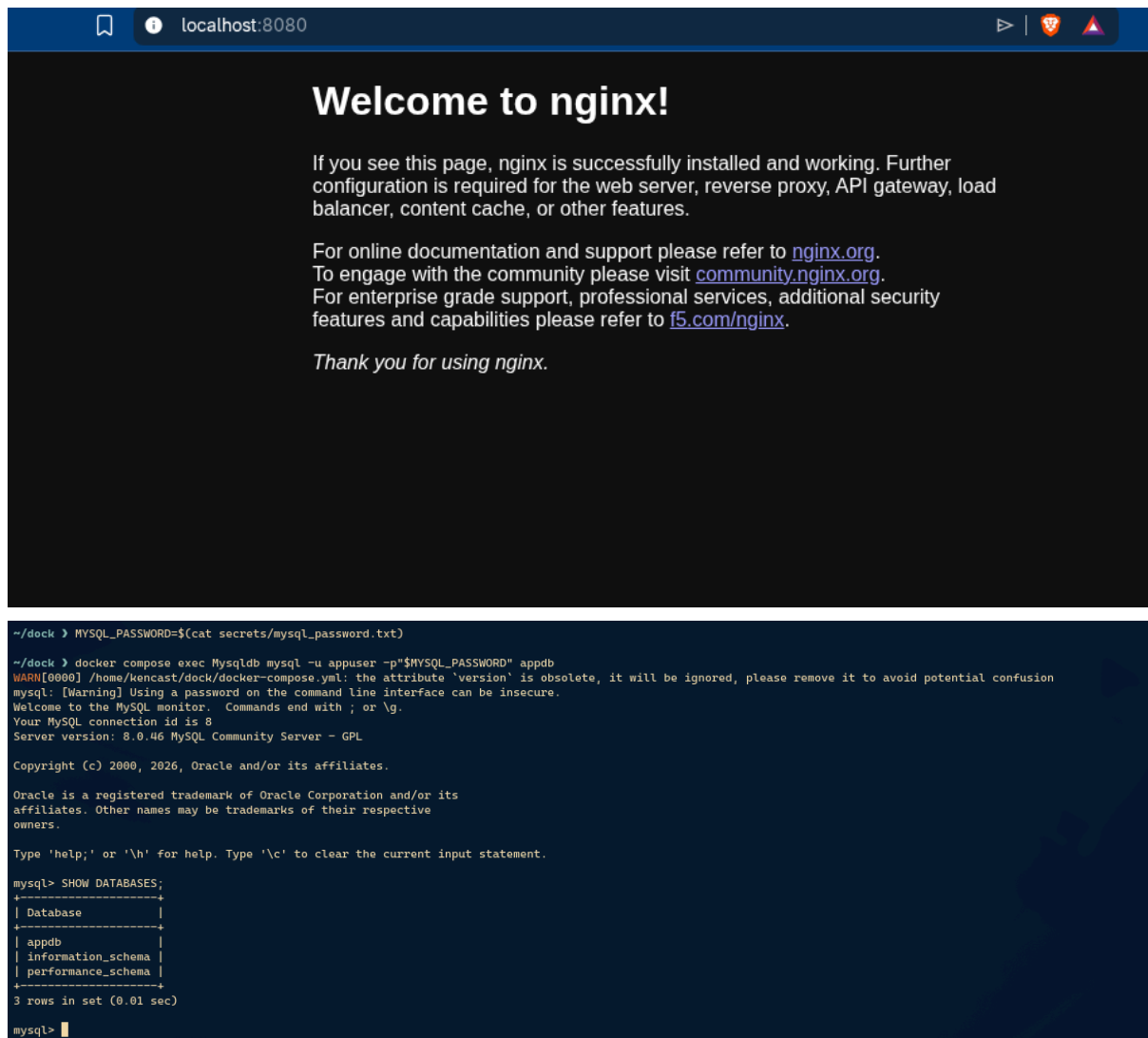
~/dock > n docker-compose.yml

~/dock > docker compose up -d
WARN[0000] /home/kencast/dock/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] up 24/24
✓ Image mysql:8.0      Pulled
✓ Image nginx:latest   Pulled
✓ Network dock_default Created
✓ Volume dock_db_data   Created
✓ Container Mysqldb     Started
✓ Container webserver    Started

~/dock > docker compose ps
WARN[0000] /home/kencast/dock/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
NAME          IMAGE          COMMAND                  SERVICE    CREATED          STATUS          PORTS
Mysqldb       mysql:8.0      "docker-entrypoint.s..." Mysqldb    31 seconds ago   Up 30 seconds   0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp, 33060/tcp
webserver     nginx:latest   "/docker-entrypoint..." webserver   31 seconds ago   Up 30 seconds   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp

~/dock > █
```

```
1  version: "3.8"
2
3  services:
4    webserver:
5      image: nginx:latest
6      container_name: webserver
7      ports:
8        - "8080:80"
9      depends_on:
10       - Mysqldb
11
12    Mysqldb:
13      image: mysql:8.0
14      container_name: Mysqldb
15      restart: always
16      ports:
17        - "3306:3306"
18      environment:
19        MYSQL_DATABASE: appdb
20        MYSQL_USER: appuser
21        MYSQL_PASSWORD_FILE: /run/secrets/mysql_password
22        MYSQL_ROOT_PASSWORD_FILE: /run/secrets/mysql_root_password
23      secrets:
24        - mysql_password
25        - mysql_root_password
26      volumes:
27        - db_data:/var/lib/mysql
28
29  secrets:
30    mysql_password:
31      file: ./secrets/mysql_password.txt
32    mysql_root_password:
33      file: ./secrets/mysql_root_password.txt
34
35  volumes:
36    db_data:
```



The image shows a web browser window with the address bar set to localhost:8080. The page displays a "Welcome to nginx!" message, stating that nginx is successfully installed and working. It provides links for online documentation (nginx.org), community support (community.nginx.org), and enterprise support (f5.com/nginx). A "Thank you for using nginx." message is also present.

Below the browser window, a terminal window shows the following commands and output:

```
~/dock > MYSQL_PASSWORD=$(cat secrets/mysql_password.txt)

~/dock > docker compose exec Mysqldb mysql -u appuser -p"$MYSQL_PASSWORD" appdb
WARN[0000] /home/kencast/dock/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SHOW DATABASES;
+-----+
| Database |
+-----+
| appdb    |
| information_schema |
| performance_schema |
+-----+
3 rows in set (0.01 sec)

mysql>
```

(13) En el archivo docker-compose.yml que usted elabora se utilizan secrets para inyectar las contraseñas de MySQL. Describa en qué consisten los secrets en docker-compose, dónde quedan disponibles dentro del contenedor (ruta), y por qué es preferible este mecanismo en lugar de definir la contraseña directamente como variable de entorno en texto plano.

Los secrets en docker-compose consisten en archivos con información sensible como contraseñas que se quiere que solo vea por los servicios que lo ocupan. La ruta en donde quedan disponibles es “/run/secrets”. Es preferible este mecanismo porque, al definir la contraseña como texto plano queda expuesta a cualquiera que acceda al archivo “docker-compose.yml”, el cual es un archivo al que se ocupa tener acceso.

(14) En la sección volumes del docker-compose.yml se declara un volumen llamado db_data. ¿Por qué se crea este volumen y para qué se usa? ¿Qué pasaría con los datos almacenados si se ejecuta docker-compose down sin la bandera -v vs. con la bandera -v?

Se crea este volumen para tener una base de datos persistente, en otras palabras, para que los datos queden guardados en el sistema host aun cuando se detenga la pila. Sin la bandera “-v”, los datos quedan guardados, mientras que con la bandera “-v” se borra la base de datos persistente.

(15) La definición de puertos del servidor web aparece como "80:80". ¿Qué significa exactamente esta sintaxis? Identifique cuál número corresponde al puerto del anfitrión y cuál al puerto del contenedor, y explique por qué se pueden mapear puertos diferentes (por ejemplo "8090:443").

El primer número corresponde al puerto del anfitrión y el segundo al del contenedor. Uno puede definir puertos otros puertos disponibles.

(16) Si en lugar de utilizar docker-compose se quisiera ejecutar únicamente la imagen mysql:5.7 desde la línea de comandos con docker run, ¿cómo le enviaríamos las variables de ambiente MYSQL_ROOT_PASSWORD, MYSQL_DATABASE, MYSQL_USER y MYSQL_PASSWORD? Escriba el comando completo equivalente.

Las variables de entorno se enviarían con la bandera “-e”:

```
docker run -d \
--name Mysqldb \
-p 3306:3306 \
-v db_data:/var/lib/mysql \
-e MYSQL_ROOT_PASSWORD=root_password \
-e MYSQL_DATABASE=appdb \
-e MYSQL_USER=appuser \
-e MYSQL_PASSWORD=app_password \
mysql:5.7
```

(17) ¿Cuál es la diferencia entre ejecutar docker-compose up y ejecutar varios docker run de forma manual? Mencione al menos tres aspectos en los que docker-compose añade valor (red interna entre servicios, orquestación declarativa, manejo de dependencias, secretos, volúmenes con nombre, etc.) y un escenario donde tendría sentido usar solamente docker run.

Docker-compose automatiza todo el proceso y flujo para correr distintos contenedores. Con docker run es un proceso mucho más lento y propenso a errores conforme crece el proyecto. Aspectos en los que docker-compose añade valor:

- Manejo de dependencia entre contenedores y parámetros.
- Manejo de datos sensibles como contraseñas con secretos.
- Orquestación y eliminación.

Tendría sentido usar solamente docker run cuando se trata solo de una imagen y no hay mucha dependencia de datos y parámetros.

(18) Con base en los principios de Seguridad del Software estudiados en el curso (mínimo privilegio, defensa en profundidad, fail-secure, separación de responsabilidades, gestión de secretos, etc.), ejemplifique con al menos cinco implementaciones concretas aplicables a un entorno con Docker: por ejemplo, no correr contenedores como root, usar imágenes oficiales o firmadas, limitar capacidades (--cap-drop), usar redes privadas, mantener imágenes actualizadas, escanear vulnerabilidades con docker scan, etc.

- Crear un usuario para correr los contenedores en vez de usar root por defecto (mínimo privilegio).
- No exponer puertos si no son necesarios (mínimo privilegio).
- No exponer contraseñas o archivos sensibles (gestión de secretos).
- Mantener las imágenes y servicios actualizados.
- Manejo correcto de recursos y memoria, eliminando datos persistentes cuando es necesario.